Reflection (individual)
Take a few minutes to reflect on your personal workflow, the concepts you encountered, and the tools you used to complete this assignment.
1. How difficult was this assignment on a scale of 1 (very easy) – 5 (very challenging)? 3
2. What was easy about this assignment and why? Understanding what was being asked was pretty easy because its not that hard of a concept.
3. What was challenging about this assignment and why? Working through actually implementing the code and getting it running functionally through my burnout was quite the task and the offensive code was neat.
4. What external tools or resources did you use to complete this assignment and where were they helpful? ChatGPT was very helpful getting the code working and running just because I have no energy to work through it on my own entirely right now. I know I was supposed to find a team I didn't

a. Names of all teammates: First and last.
Jesse Martin

Service Overview

Our team selected the **Network Sentinel / Firewall Agent** as the security domain for our prototype. The service acts as a lightweight firewall that receives simulated network packet requests and determines whether incoming traffic should be allowed or blocked based on a predefined blacklist of IP addresses. The service processes requests using UDP sockets and JSON messages, returning either an `ALLOW` or `BLOCK` response depending on the source IP address.

To simulate an unpredictable system failure, the Network Sentinel implements a randomized crash mechanism. Each incoming datagram has a 2% probability of triggering a fatal process termination using Python's `os._exit(1)` function. This simulates unexpected software failures and ensures that the entire service terminates rather than simply becoming unresponsive temporarily. An independent heartbeat monitor detects these failures by periodically checking whether the service is responding. The primary objective of the prototype is to demonstrate how heartbeat-based monitoring can detect critical service failures and how weaknesses in heartbeat communication can affect monitoring reliability.

Execution & Setup Instructions

The prototype consists of three Python scripts: `sentinel.py`, `monitor.py`, and `heartbeat_spoof_audit.py`. Python 3.10 or later is required. All scripts use Python's standard library, so no additional packages or compilation steps are necessary.

Step 1: Navigate to the project directory.

Open a terminal and navigate to the directory containing the Python files.

cd network_sentinel

Step 2: Start the Network Sentinel service.

In the first terminal, execute:

python sentinel.py

This starts the Network Sentinel on UDP port `39001` using the localhost address `127.0.0.1`. The service listens for simulated packet inspection requests and heartbeat messages.

For testing purposes, the crash probability can be increased to 20%:

python sentinel.py --failure-probability 0.20

This increases the likelihood of observing a process failure during a short demonstration.

Step 3: Start the heartbeat monitor.

Open a second terminal in the same directory and execute:

python monitor.py

The monitor sends a `PING` message to the Network Sentinel every second and waits for an acknowledgment. If the monitor fails to receive three consecutive acknowledgments, it reports that the service is unresponsive.

The monitor can also be started with explicit timing parameters:

python monitor.py --interval 1 --timeout 2 --misses 3

Detected failures are printed to the terminal with timestamps and recorded in a file named `monitor.log`.

Step 4: Observe the simulated failure.

Allow both processes to run until the Network Sentinel experiences a randomized crash. Once the service terminates, the monitor will begin reporting missed heartbeats.

After three consecutive failed heartbeat attempts, the monitor generates an alert and records the incident. The monitor continues running independently after the service has terminated.

Step 5: Execute the adversarial assessment.

Open a third terminal in the project directory.

Stop any existing monitor instances before starting the standalone adversarial assessment to avoid interfering with the test.

Execute:

python heartbeat_spoof_audit.py

The attack assessment launches a separate monitor instance and uses a simulated responder to transmit forged heartbeat acknowledgments. The actual Network Sentinel does not need to be running for this assessment.

The test evaluates whether the monitor accepts these acknowledgments and incorrectly reports that the service is healthy.

Step 6: Review the results.

The terminal output indicates whether the monitor accepted the simulated heartbeat acknowledgments.

The normal monitoring process records detected failures in:

monitor.log

The adversarial assessment can be considered successful if the monitor reports successful heartbeats despite the legitimate Network Sentinel not running.

Architectural Parameter Justification

i. Heartbeat Pattern: Pull/Ping-Echo

Our team selected **Option B: Pull/Ping-Echo** for the heartbeat monitoring mechanism.

Under this architecture, the independent monitor periodically sends a `PING` message to the Network Sentinel. When the service receives the message, it responds with an `ACK` acknowledgment. The monitor uses the presence or absence of these acknowledgments to determine whether the service is operational.

We selected this approach because it gives the monitor direct control over the frequency of health checks. Unlike a push-based heartbeat system, where the service is responsible for periodically transmitting heartbeat messages, the pull-based approach allows the monitor to initiate communication and measure whether the service responds within an acceptable period.

Additionally, the pull-based architecture allows the monitor to remain independent of the critical service. If the Network Sentinel crashes, the monitor continues sending heartbeat requests and can identify the failure without relying on the service to report its own status.

UDP was selected because it provides lightweight communication with minimal overhead. However, UDP does not guarantee message delivery, making appropriate timeout thresholds necessary.

ii. Timing Calibration

Our team configured the heartbeat monitoring system with the following parameters:

Parameter	Value	Description
Heartbeat Interval (T)	1 second	The monitor attempts to send a heartbeat every second.
Timeout Window (W)	2 seconds	Maximum time the monitor waits for an acknowledgment.
Missed-Beat Threshold (N)	3	Number of consecutive missed acknowledgments required to declare a failure.

Heartbeat Interval (T = 1 second):

A one-second heartbeat interval was selected to provide frequent service availability checks without generating excessive network traffic. Because heartbeat messages are small UDP datagrams, the communication overhead is minimal.

Timeout Window (W = 2 seconds):

A two-second timeout provides the service with sufficient time to respond while accounting for potential processing delays or temporary network congestion. This helps prevent the monitor from interpreting every delayed response as a service failure.

Missed-Beat Threshold (N = 3):

Requiring three consecutive missed acknowledgments reduces the likelihood of false-positive failure alerts caused by occasional packet loss or temporary communication delays. A single missed heartbeat does not immediately indicate that the service has crashed.

Mean Time to Detect (MTTD):

The selected parameters balance detection speed against monitoring reliability.

Because the current implementation waits up to two seconds for each unanswered heartbeat, three consecutive timeouts produce a failure alert approximately six seconds after the first unsuccessful heartbeat attempt.

The actual detection time depends on when the service crashes relative to the monitor's heartbeat schedule. The implementation waits for each heartbeat attempt to finish before

beginning the next, so the effective interval between attempts may exceed one second when requests time out.

These parameters were selected because a detection delay of several seconds is acceptable for a simulated firewall prototype. The configuration provides relatively fast failure detection while reducing unnecessary alerts caused by isolated communication failures.

Adversarial Audit Summary

For our adversarial assessment, we selected **Attack 1: Heartbeat Spoofing**. The objective was to determine whether an attacker could manipulate the heartbeat monitoring mechanism into reporting that the Network Sentinel was operational even when the legitimate service was unavailable. The baseline heartbeat protocol uses UDP messages containing simple JSON fields, including `"type": "PING"` and `"type": "ACK"`. However, the monitor only checks whether the received response contains the expected acknowledgment type. It does not authenticate the sender or verify that the acknowledgment corresponds to the current heartbeat request. Consequently, the monitoring mechanism is vulnerable to accepting forged heartbeat responses because it cannot reliably distinguish a legitimate service response from an attacker-generated acknowledgment.

To assess this vulnerability, we developed `heartbeat_spoof_audit.py`, which uses a simulated responder to generate fake heartbeat acknowledgments while the legitimate Network Sentinel is not running. In the local test, the monitor accepted the simulated acknowledgments and reported successful heartbeats despite the absence of the legitimate service. This demonstrated that the baseline protocol could be deceived by an unauthorized responder. The test successfully demonstrated the authentication weakness using a controlled local simulation, although it did not establish that an external attacker could inject packets into an already-running monitor session. To mitigate this vulnerability, the heartbeat protocol should incorporate cryptographic message authentication, such as HMAC, and unique challenge-response values to prevent unauthorized devices or processes from impersonating the monitored service. These improvements would strengthen the reliability of the monitoring mechanism by ensuring that heartbeat acknowledgments originate from an authorized source and correspond to legitimate monitoring requests.